

```
"""
LifeLens AI – AI-powered health
awareness and medical education
Cloud-compatible version (Streamlit
Community Cloud + Google Gemini API)

Originally based on the MedGuide AI
concept (local Ollama + Llama 3.2
3B).
This version replaces the local
Ollama backend with the Gemini API so
it
works when deployed on Streamlit
Community Cloud, which cannot reach a
local Ollama service.
"""

import os
import sqlite3
from datetime import datetime

import streamlit as st
from google import genai

# -----
-----
```

```

---
# Config
# -----
-----
---
MODEL_NAME = "gemini-3.5-flash" #
current stable Gemini model (2026)
DB_PATH = "lifelens_history.db"

st.set_page_config(page_title="LifeLe
ns AI", page_icon="🦋",
layout="wide")

# -----
-----
---
# API key handling – Streamlit
Secrets first, env var fallback.
Never hardcode.
# -----
-----
---
def get_api_key():
    try:
        key =
st.secrets["GEMINI_API_KEY"]
        if key:
            return key

```

```
        except Exception:
            pass
        return
    os.getenv("GEMINI_API_KEY")

@st.cache_resource
def get_client():
    api_key = get_api_key()
    if not api_key:
        return None
    return
    genai.Client(api_key=api_key)

client = get_client()

# -----
# -----
# -----
# SQLite history (demo/local – resets
# when the Cloud app's filesystem
# resets)
# -----
# -----
# -----
def init_db():
    conn = sqlite3.connect(DB_PATH)
```

```

conn.execute(
    """
        CREATE TABLE IF NOT EXISTS
history (
            id INTEGER PRIMARY KEY
AUTOINCREMENT,
            timestamp TEXT,
            category TEXT,
            question TEXT,
            answer TEXT
        )
    """
)
conn.commit()
conn.close()

def save_history(category, question,
answer):
    conn = sqlite3.connect(DB_PATH)
    conn.execute(
        "INSERT INTO history
(timestamp, category, question,
answer) VALUES (?, ?, ?, ?)",
        (datetime.now().strftime("%Y-
%m-%d %H:%M"), category, question,
answer),
    )
    conn.commit()

```

```

conn.close()

def load_history():
    conn = sqlite3.connect(DB_PATH)
    rows = conn.execute(
        "SELECT timestamp, category,
question, answer FROM history ORDER
BY id DESC"
    ).fetchall()
    conn.close()
    return rows

init_db()

# -----
-----
---
# Gemini call
# -----
-----
---
SYSTEM_INSTRUCTION_TEMPLATE = """You
are LifeLens AI, an educational
health-awareness assistant.

Answer in {language}.
```

Use a {level} explanation level.

Give general educational health information only.

Do not diagnose a person or claim certainty about a medical condition.

Do not prescribe medicines or medication doses.

If symptoms may be urgent, clearly recommend contacting a qualified healthcare professional or local emergency service.

Keep answers understandable and practical."""

```
def ask_gemini(system_instruction:
str, prompt: str) -> str:
    if client is None:
        return (
            "⚠️ No Gemini API key
found. Add `GEMINI_API_KEY` in
Streamlit "
            "Secrets (see setup
instructions) and reload the app."
        )
    try:
        response =
```

```

client.models.generate_content(
    model=MODEL_NAME,
    contents=prompt,
    config=
{"system_instruction":
system_instruction},
    )
    return response.text
except Exception as e:
    return f"⚠️ Could not get a
response from Gemini right now.
Error: {e}"

# -----
-----
---
# Sidebar settings
# -----
-----
---
st.title("🩺 LifeLens AI")
st.caption("AI-powered health
awareness and medical education – for
learning, not diagnosis.")

with st.sidebar:
    st.header("Settings")
    language =

```

```

st.selectbox("Language", ["English",
"Tamil"])
    level = st.selectbox("Explanation
level", ["Simple", "Student",
"Detailed"])
    st.markdown("---")
    st.caption(
        "LifeLens AI (based on the
MedGuide AI concept) · "
        f"Model: `{MODEL_NAME}`"
    )
    if client is None:
        st.error("Gemini API key not
configured yet.")

system_instruction =
SYSTEM_INSTRUCTION_TEMPLATE.format(la
nguage=language, level=level)

tab1, tab2, tab3, tab4 = st.tabs(
    ["🩺 Health Awareness", "📖
Learn", "🚨 Urgency Check", "🕒
History"]
)

# -----
-----
---
```

```

# Tab 1: Health Awareness
# -----
---
with tab1:
    st.subheader("Health Awareness")
    question = st.text_area("What
would you like to understand?",
key="ha_input")
    if st.button("Explain",
key="ha_button"):
        if question.strip():
            with
st.spinner("Thinking..."):
                answer =
ask_gemini(system_instruction,
question)
                st.markdown(answer)
                save_history("Health
Awareness", question, answer)
            else:
                st.warning("Please enter
a question first.")

# -----
---
# Tab 2: Learn

```

```

# -----
-----
---
with tab2:
    st.subheader("Learn")
    topic = st.text_input("Enter a
topic", key="learn_input")
    if st.button("Create Study Pack",
key="learn_button"):
        if topic.strip():
            learn_prompt = f"""Create
a study pack about: {topic}

Structure your response with these
exact sections:
1. Short Overview (2-3 sentences)
2. Key Points (5-8 bullet points)
3. Important Terms (with brief
definitions)
4. 5 Multiple Choice Questions, each
with 4 options and the correct answer
clearly marked
5. 5 Flashcards (Term -> Definition
format)"""

            with st.spinner("Building
your study pack..."):
                answer =
ask_gemini(system_instruction,
learn_prompt)

```

```

        st.markdown(answer)
        save_history("Learn",
topic, answer)
    else:
        st.warning("Please enter
a topic first.")

# -----
-----
---
# Tab 3: Urgency Check
# -----
-----
---
with tab3:
    st.subheader("Urgency Check")
    st.info("This section is for
general awareness only. It does not
diagnose medical conditions.")
    symptoms = st.text_area("Describe
the symptoms or situation",
key="urgency_input")
    if st.button("Check Urgency",
key="urgency_button"):
        if symptoms.strip():
            urgency_prompt = f"""A
user describes the following symptoms
or situation: {symptoms}

```

Please:

- Identify general warning signs present, if any
- Explain when this type of situation may warrant professional medical attention
- Explain what would make it more urgent (i.e. emergency-level)
- Clearly recommend contacting a qualified healthcare professional or local

emergency service when appropriate

- Do NOT diagnose the user or name a specific condition with certainty""

with

```
st.spinner("Checking..."):
```

```
    answer =
```

```
ask_gemini(system_instruction,  
urgency_prompt)
```

```
    st.markdown(answer)
```

```
    save_history("Urgency  
Check", symptoms, answer)
```

```
else:
```

```
    st.warning("Please  
describe the symptoms or situation  
first.")
```

```

# -----
-----
---
# Tab 4: History
# -----
-----
---
with tab4:
    st.subheader("History")
    st.caption(
        "Local/demo history – may
reset if the app restarts on
Streamlit "
        "Community Cloud, since its
filesystem is not guaranteed to
persist."
    )
    rows = load_history()
    if not rows:
        st.write("No history yet.")
    else:
        for ts, category, q, a in
rows:
            with st.expander(f"
[{category}] {ts} – {q[:60]}"):
                st.markdown(f"Question: {q}")

                st.markdown(f"Answer: {a}")

```

